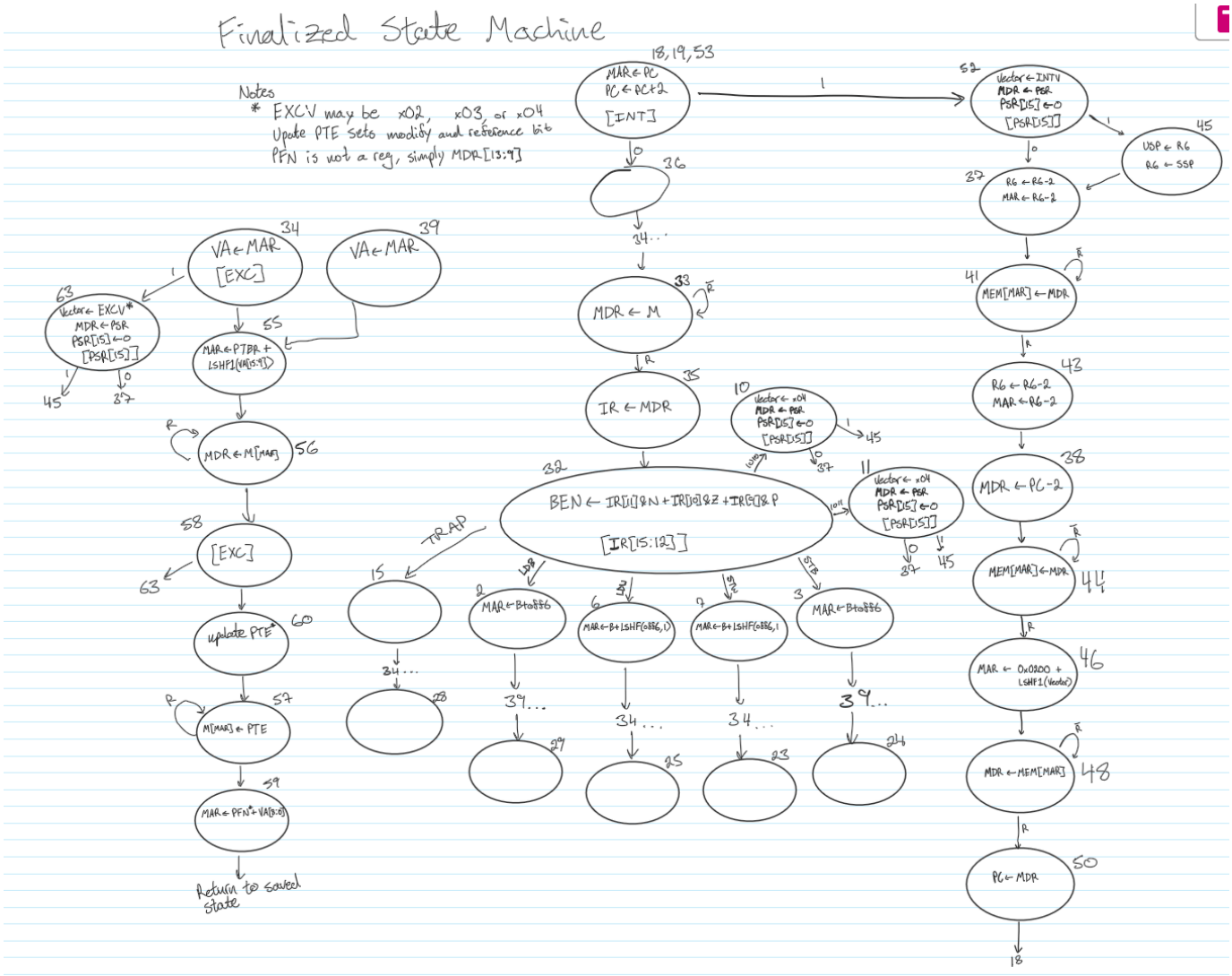
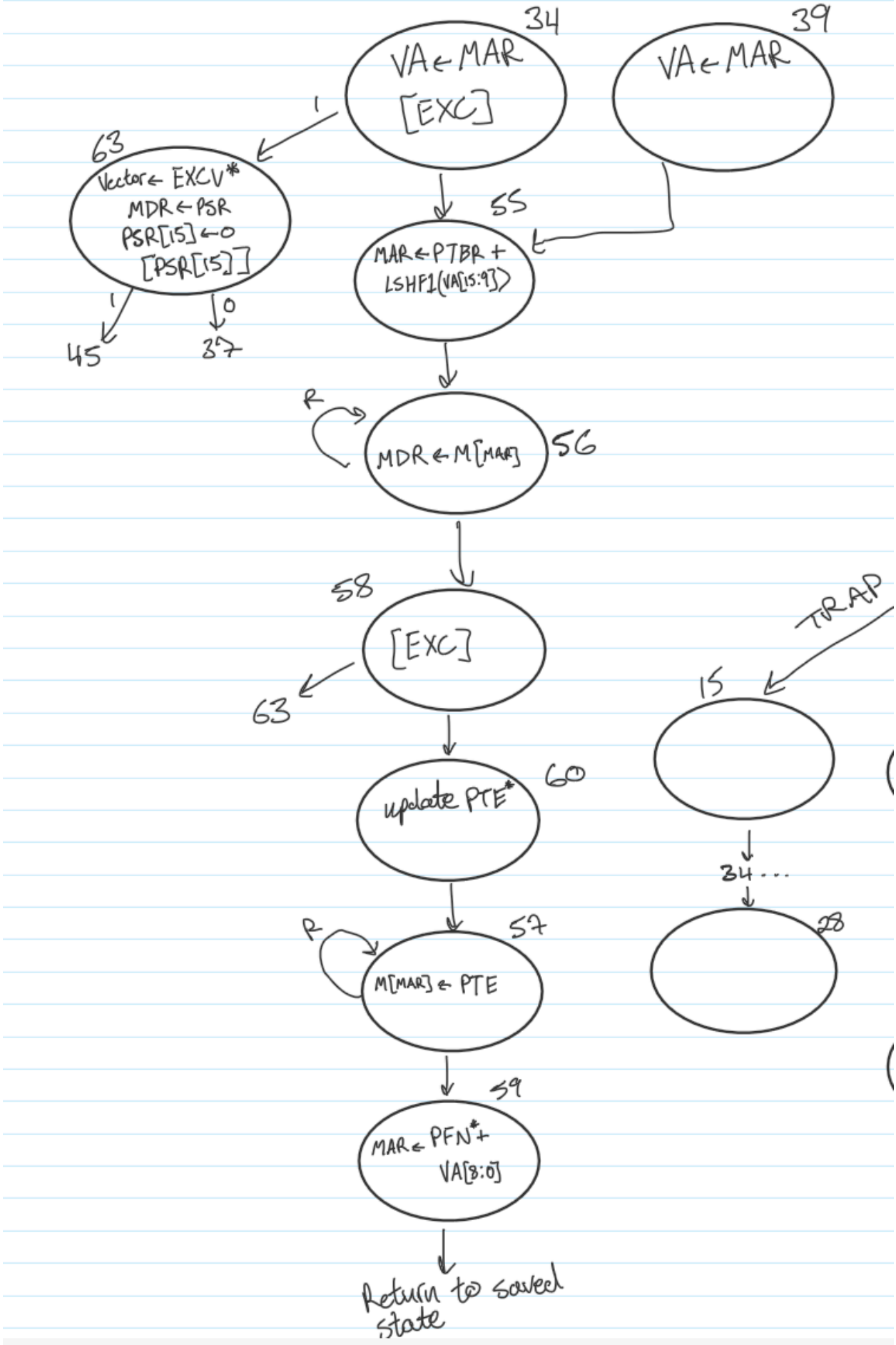
Lab 5 Documentation

Tyler Casey - TNC824

A1. State Machine



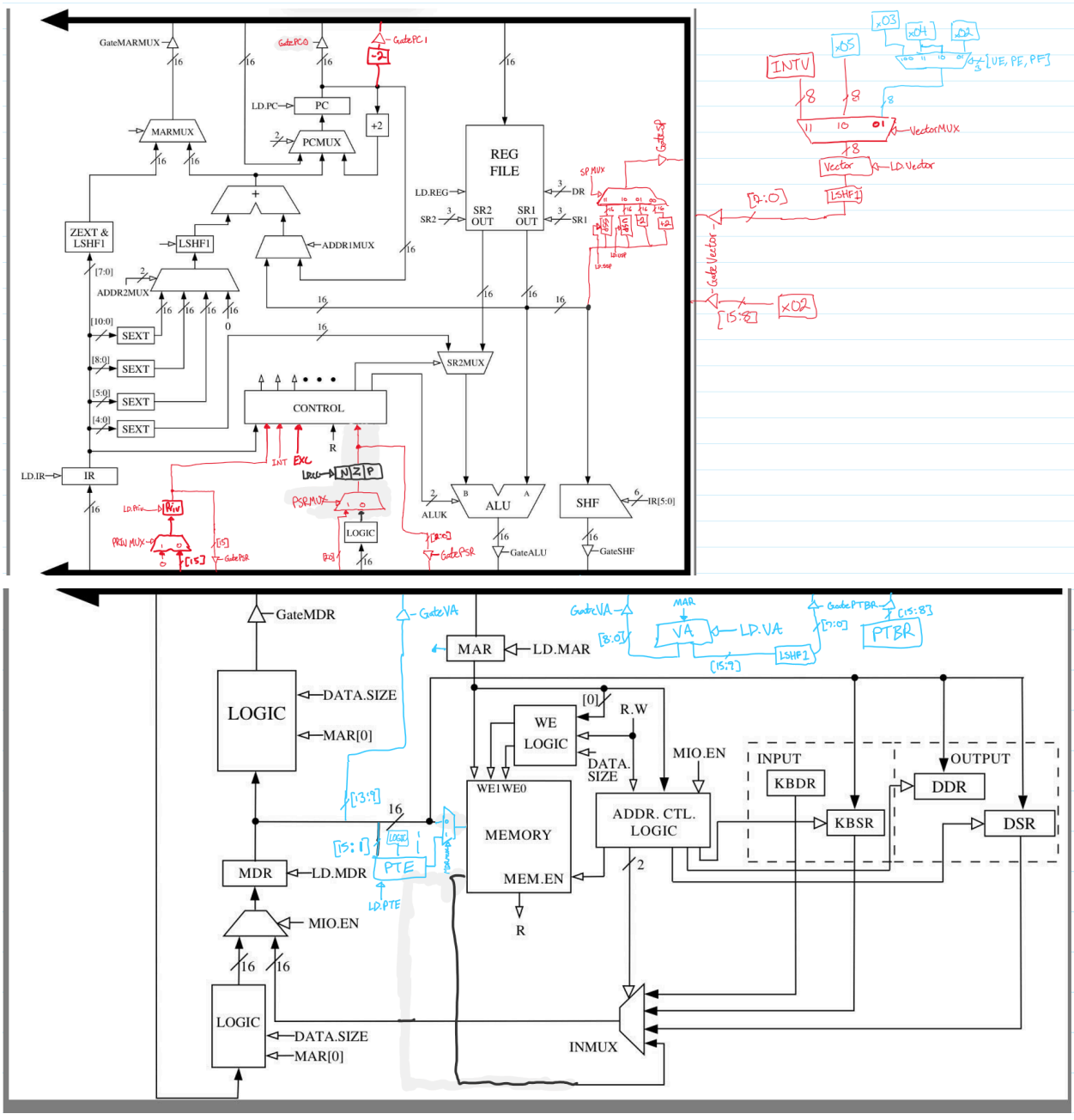

34
VA ← MAR
[EXC]
39
VA ← MAR
1
63
Vector ← EXCV*
MDR ← PSR
PSR[15] ← 0
[PSR[15]]
1
0
45
37
55
MAR ← PTBR +
LSHF1(VA[15:9])
R
MDR ← M[MAR]
56
58
[EXC]
63
TRAP
15
60
update PTE*
34 ...
R
57
M[MAR] ← PTE
28
59
MAR ← PFN* +
VA[8:0]
Return to saved state

Explanation:
There were updates and new additions to the statemachine from lab4, I will explain these in sections.
Part 1: Address Translation
Every access to memory will require the address to first be translated from its virtual address to the physical address. I created a sequence shown on the left of the image to walk through address translation. Firstly the current MAR is stored in the VA register (unaligned exception is also checked depending on whether the MAR is a word or byte), then MAR is updated to have the address of the PTE we are searching for. This address is then loaded from memory, putting the PTE into the MDR, using this value page fault and protection exceptions are checked. The PTE reference and modify bits are updated accordingly (see datapath and additional logic block section **A2** & **A3**). The PFN (MDR[13:9]) and the VA offset bits are bit back into MAR for the PA.

Part 2: Updates to the previous statemachine
In my Lab 4 implementation I had a unique exception checking state for every MAR access. However in Lab 5 I realized I could reuse states that did the same work, especially since I now have the ability to store states (see updated microsequencer section **A4**). So every state that accessed new memory I sent to state 34 (for word accesses) or 39 (byte accesses). These states will store their next state in a register and return to it after the translation sequence has completed. Additionally, I created a blank state 36 to account for previous state numbering and so that unnecessary translations wouldn't occur during interrupt signal checks.

All other updates remain the same from lab4, as well as numbering. (Lab4 material is listed in Appendix)
Total States Used: 62

A2. Datapath



Explanation:

The datapath had three main changes: exception/vector generation, physical address, and updating lab4 datapath

Vector Generation:

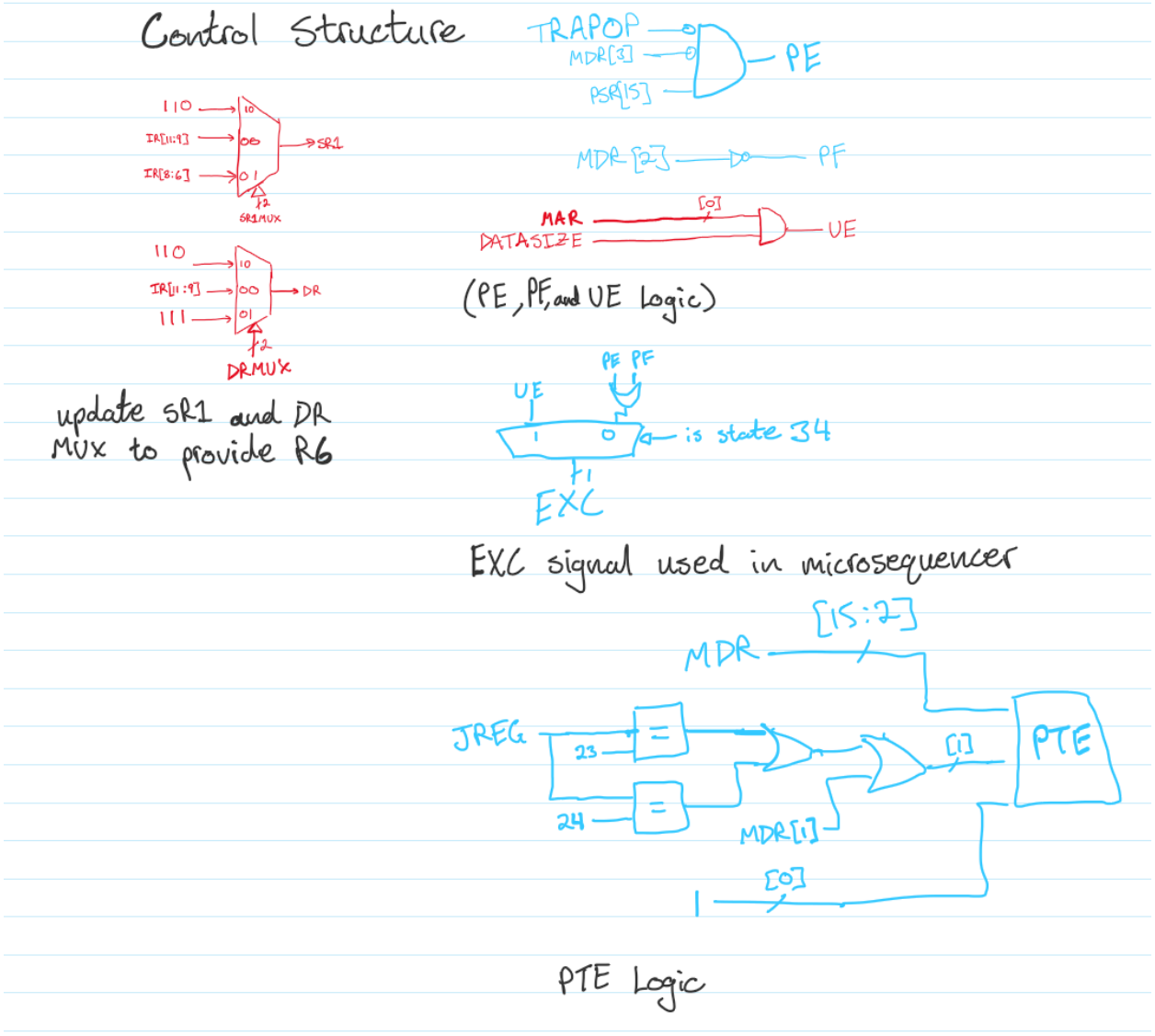
The base addition is the same as lab4 however in this lab the vector values were changed, unknown opcode is now x05, protection exception is now x04, and the new page fault exception is x02 (unaligned exception remains x03). The vectorMUX is unchanged in how it selects, however now the MUX responsible for selecting unaligned and protection now includes page fault. The three exception signals

are sent into the mux, selecting one or the other based off of priority (UE mapping to all other binary combinations not shown).

Physical Address:
In order to account for the virtual address translation to physical address, three registers were added to the datapath: VA, PTBR, and PTE. PTBR is sent onto the bus alongside left shifted VA[15:9] bits to find the address of the PTE. MAR is wired directly to VA, since VA only takes MAR values. The PTE register has logic explained in section A3 and is wired directly to memory. There is an additional MUX (MDRMUX) to select what values are being sent to memory (MDR or PTE). MDR[13:9] (PFN) is sent alongside VA[8:0] onto the bus to put the PA into MAR.

Lab 4 Updates:
I left out the drawing of exception registers, but their logic is included in the additional logic/components section.

A3. Additional Logic/Components



Explanation:
Protection exception and Page fault are now calculated based on the value in the PTE. PE is set high when the instruction is not a TRAP, MDR[3] (protection bit) is 0, and PSR[15] is 1. Page fault is set when MDR[2] (valid bit) is 0.

EXC logic is now based off of PE, UE, and PF, if the state is 34 (checking unaligned) only UE will be sent to signal the EXC. Otherwise PE or PF will set the EXC signal.

PTE logic is based on the stored state, if the stored state is a 23 or 24 (the next states for ST instructions) bit 1 of the PTE (Modify bit) will be set high, otherwise it will remain the value that it was beforehand. Bit 0 (reference bit) will always be set to 1 on an access. And all the remaining bits are based off the MDR value loaded.

A4. New Control Signals

LD Signals:

LD.JREG: Loads the next state information from the Jbit output.
LD.VA: Loads the current MAR value into the VA register
LD.PTE: Loads the current MDR value, sets bit 1 (dirty/modified) based on logic in section A3, and sets bit 0 (reference) to 1.

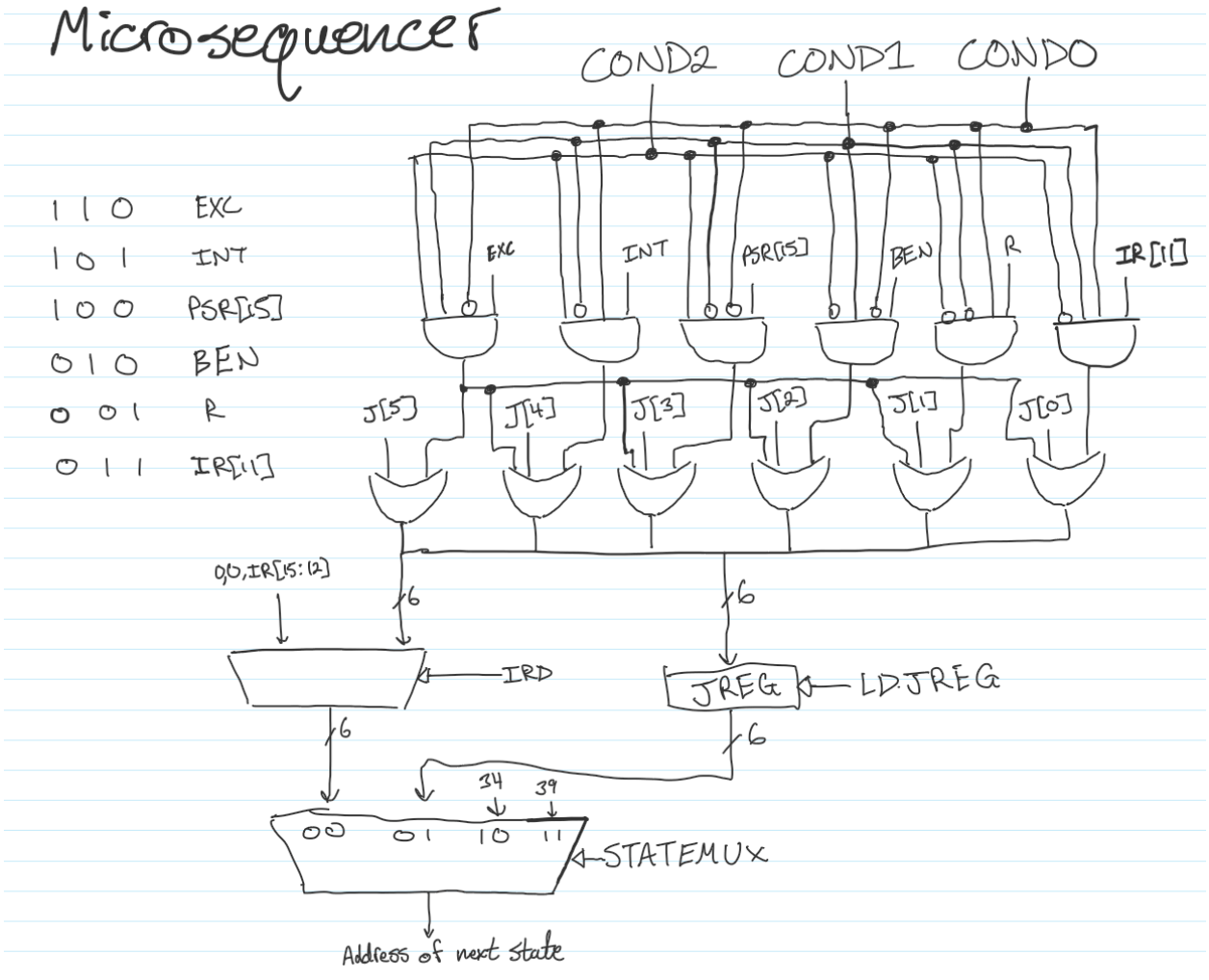
Gate Signals:

GatePTBR: GatePTBR is used for finding the address of the PTE, gates PTBR [15:8] concatenated with a left shifted VA[15:9] onto the bus (LSHF(VA[15:9]) is put on the bus as bits [7:0]).
GateVA: GateVA is used for sending the PA into the MAR, it gates MDR[13:9] (PFN) concatenated with VA[8:0] (offset bits) onto the bus.

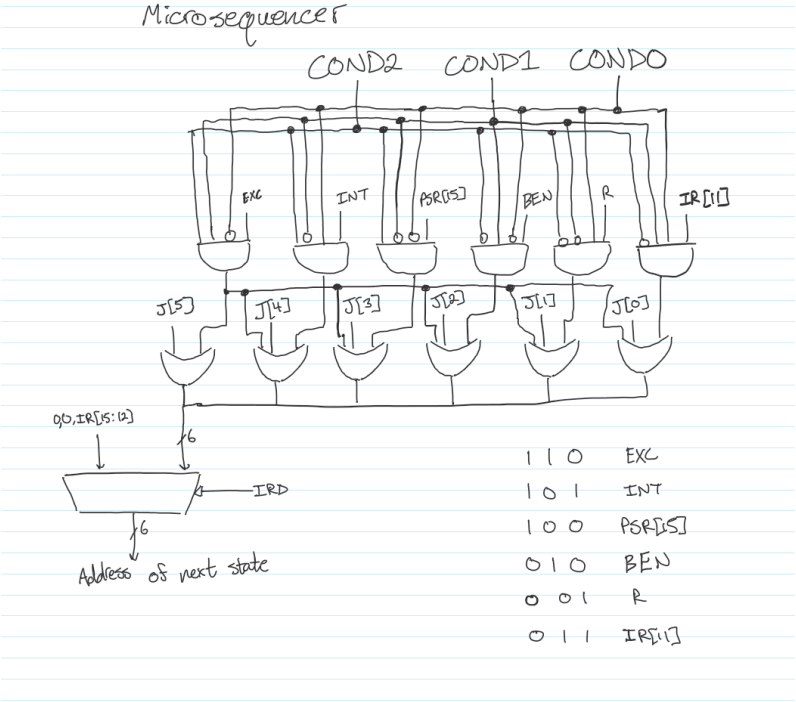
MUXES:

STATEMUX: The statemux is responsible for selecting what is outputted for the next state. (00) will send the result of the IRDMUX, (01) will send the value stored in the JREG, (10) will send state 34 (state of beginning of address translation - unaligned exception check), and (11) will send state 39 (state of beginning of address translation - no unaligned exception check).
MDRMUX: Because memory only has one input port, and I want to directly write back the PTE to memory, this required a MUX to select whether I wanted to send the value in the MDR or the value in the PTE. Therefore MDRMUX (1) will send the PTE value and a (0) will send the MDR value.

A5. Microsequencer



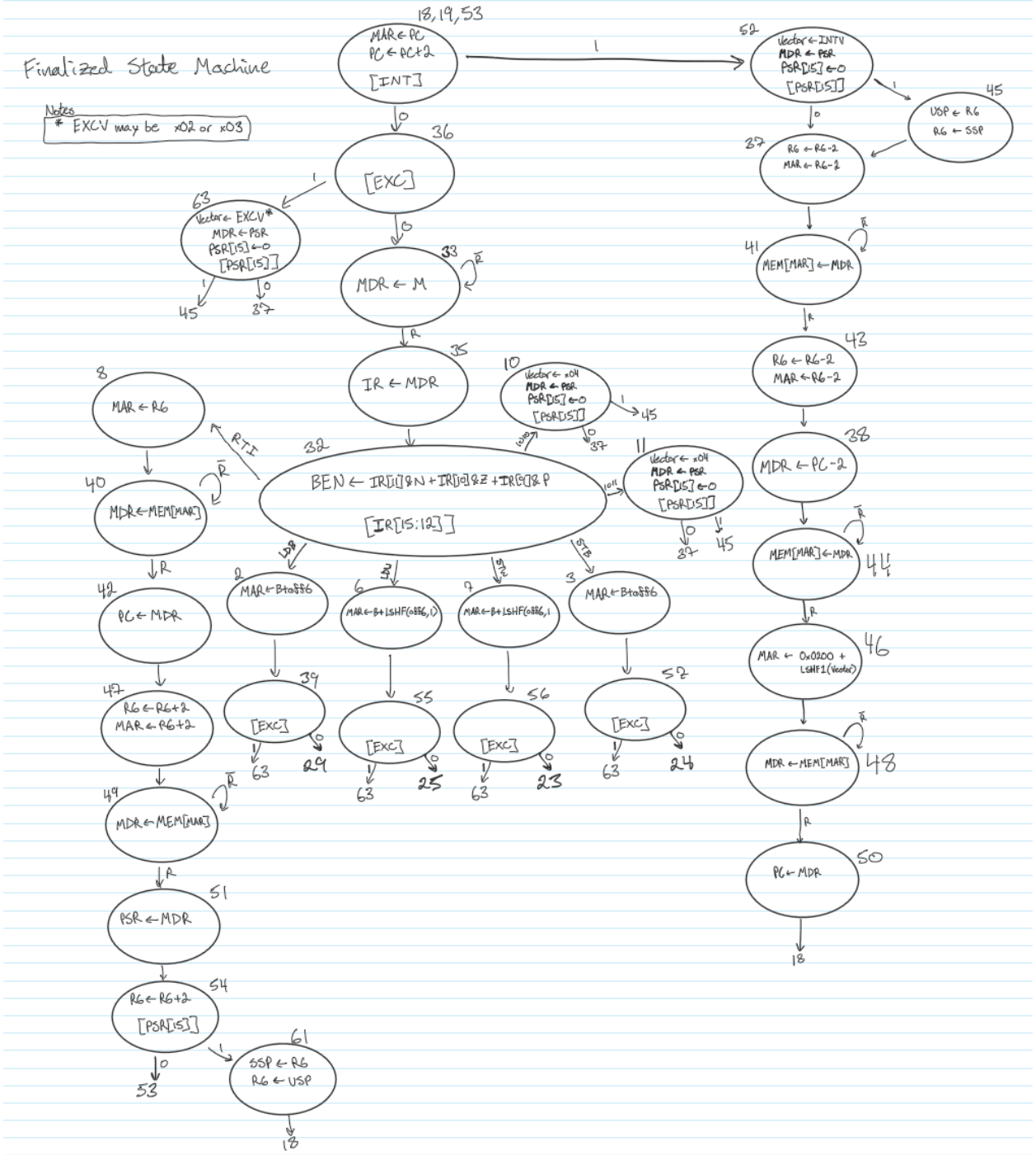
Lab4 microsequencer:

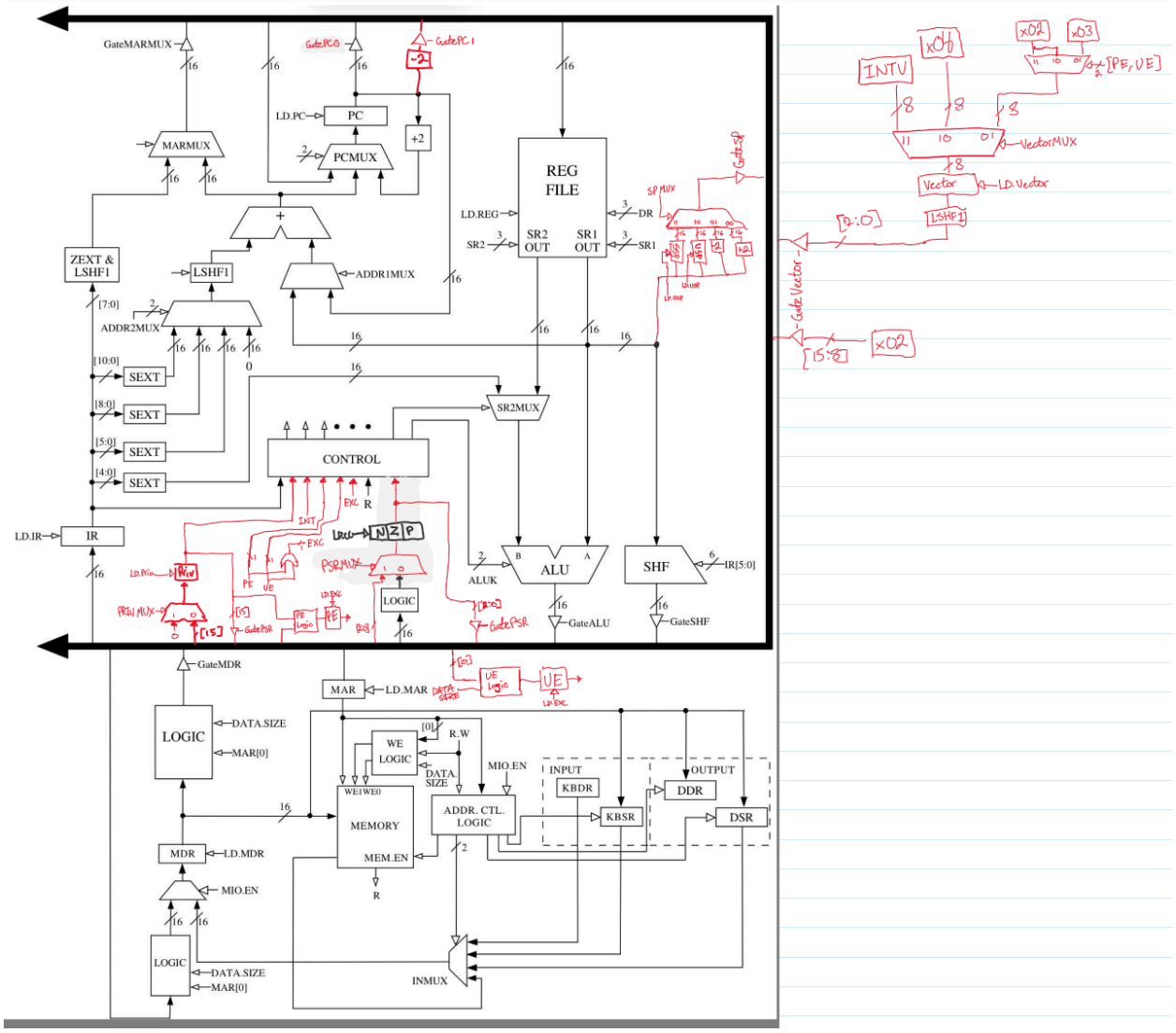


Explanation:
The Microsequencer from Lab 4 was updated so that it could store next state information. This was used when going to the address translation state sequence, allowing the return to the proper next state. This also required the addition of a stateMUX as well, selecting either the previous lab4 logic, the saved state, or either state 34 (the start of the translation sequence that also checks for unaligned exception) or state 39 (start of translation sequence when not needing to check unaligned exception).

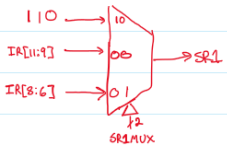
APPENDIX

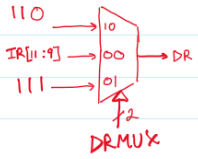
Lab 5 notes and scratch work: Lab 5.pdf
Lab 4 information: Lab 4 Documentation
Pictures from this are included below


Finalized State Machine
Notes
* EXCV may be x02 or x03
18,19,53
MAR ← PC
PC ← PC+2
[INT]
52
Vector ← INTV
MDR ← PSR
PSR[15] ← 0
[PSR[15]]
45
USP ← R6
R6 ← SSP
36
[EXC]
63
Vector ← EXCV*
MDR ← PSR
PSR[15] ← 0
[PSR[15]]
37
R6 ← R6-2
MAR ← R6-2
33
MDR ← M
41
MEM[MAR] ← MDR
35
IR ← MDR
43
R6 ← R6-2
MAR ← R6-2
8
MAR ← R6
10
Vector ← x04
MDR ← PSR
PSR[15] ← 0
[PSR[15]]
38
MDR ← PC-2
RTI
32
BEN ← IR[11]&N + IR[10]&Z + IR[9]&P
[IR[15:12]]
11
Vector ← x04
MDR ← PSR
PSR[15] ← 0
[PSR[15]]
40
MDR ← MEM[MAR]
MEM[MAR] ← MDR
44
LDB
STB
42
PC ← MDR
2
MAR ← B+x0F86
6
MAR ← B+LSHF(x0F86,1)
7
MAR ← B+LSHF(x0F86,1
3
MAR ← B+x0F86
46
MAR ← x0200 + LSHF1(Vector)
47
R6 ← R6+2
MAR ← R6+2
39
[EXC]
55
[EXC]
56
[EXC]
52
[EXC]
MDR ← MEM[MAR]
48
49
MDR ← MEM[MAR]
50
PC ← MDR
51
PSR ← MDR
54
R6 ← R6+2
[PSR[15]]
61
SSP ← R6
R6 ← USP
53
18

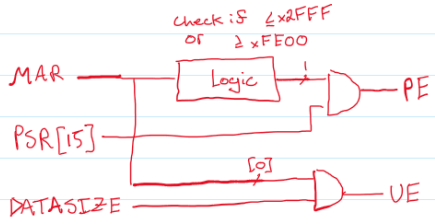

GateMARMUX
GatePC.0
GatePC.1
-2
LD.PC
PC
+2
MARMUX
PCMUX
REG FILE
LD.REG
SR2 OUT
SR1 OUT
DR
SR1
SR2
GateSP
SP MUX
ZEXT & LSHF1
LSHF1
ADDR1MUX
ADDR2MUX
[7:0]
[10:0]
[8:0]
[5:0]
[4:0]
SEXT
SR2MUX
CONTROL
INT
EXC
R
LD.IR
IR
PSR MUX
LOGIC
ALUK
ALU
SHF
IR[5:0]
GateALU
GateSHF
GatePSR
GateMDR
MAR
LD.MAR
DATA.SIZE
MAR[0]
WE LOGIC
R.W
MIO.EN
INPUT
OUTPUT
KBDR
DDR
KBSR
DSR
WE1 WE0
MEMORY
MEM.EN
ADDR. CTL. LOGIC
MDR
LD.MDR
INMUX
x01
x02
x03
INTV
[PE, UE]
VectorMUX
Vector
LD.Vector
LSHF1
GateVector
[7:0]
[15:8]
x02

Control Structure



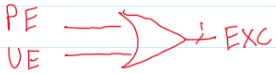


update SR1 and DR MUX to provide R6



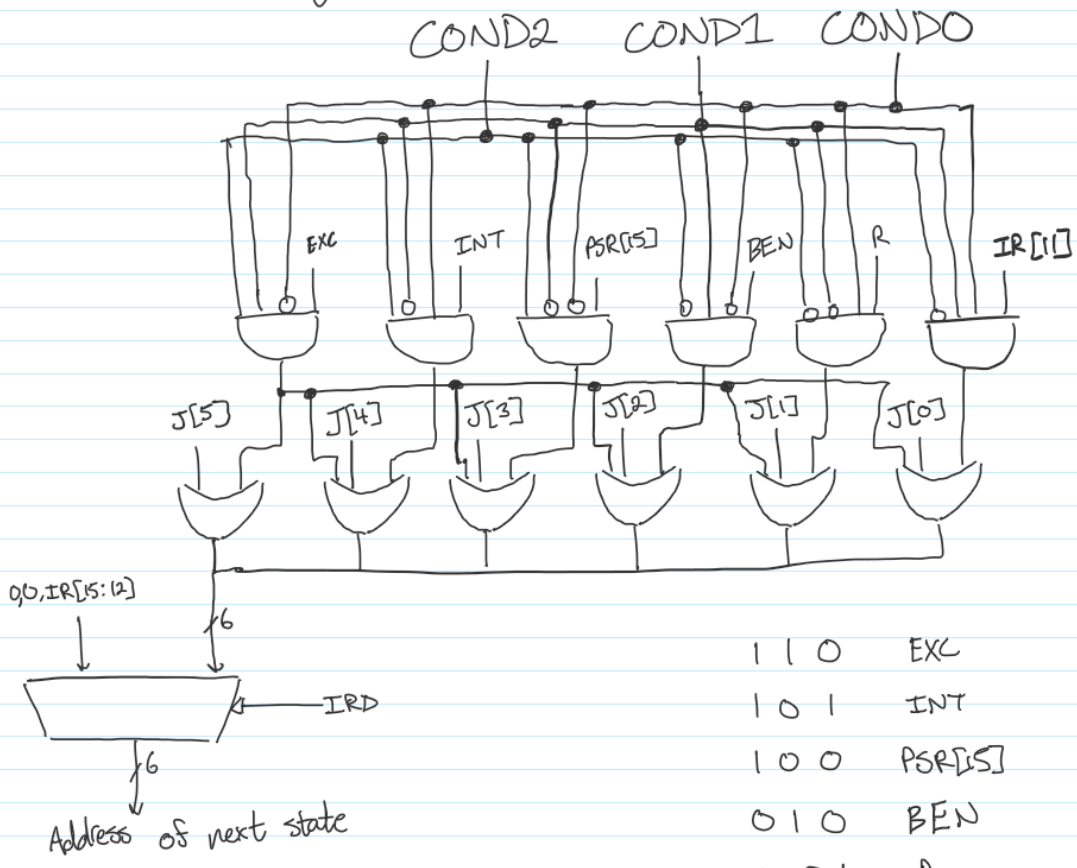
(PE and UE logic)

note that PE and UE have designated registers in datapath, this is just the logic



EXC signal used in microsequencer

Microsequencer



1 1 0	EXC
1 0 1	INT
1 0 0	PSR[15]
0 1 0	BEN
0 0 1	R
0 1 1	IR[11]